

NECCDC 2026 — Runbook 1 of 3

DC01 Full Hardening Runbook

Assume DC01 is compromised until proven otherwise.

Covers every PowerShell command needed from first login through full hardening. Follow sections in order.

Phase 0 — Assume Compromise Mindset

You have not been on this DC. Treat every account, every service, every scheduled task as potentially backdoored. Do not trust anything until you have verified it yourself. Work fast but methodically.

■ Do NOT use any cached credentials from DC02 to log into DC01 until you have verified DC01 is clean. Red Team may have a keylogger or credential capture running.

First Commands — Who Are You and What Is This Machine

```
whoami
whoami /groups
hostname
(Get-WmiObject Win32_ComputerSystem).Domain
ipconfig /all
Get-ADDomain | Select DNSRoot, PDCEmulator, Forest
Get-ADDomainController -Filter * | Select Name, IPv4Address, Site
```

Immediately Check Replication Health

```
repadmin /showrepl
repadmin /replsummary
nltest /dsgetdc:chefops.local
```

■ If replication is broken between DC01 and DC02, authentication across the domain becomes unreliable. Fix replication before hardening users.

Phase 1 — Persistence Hunt (Do This Before Anything Else)

Red Team has had uncontested access to this DC. Hunt persistence before you change passwords or harden anything — otherwise you may harden around a backdoor and miss it entirely.

1.1 — Scheduled Tasks

```
# All non-Microsoft tasks
Get-ScheduledTask |
  Where-Object {$_.TaskPath -notlike '\Microsoft\*'} |
  Select TaskName, TaskPath, State

# What each suspicious task actually executes
Get-ScheduledTask |
  Where-Object {$_.TaskPath -notlike '\Microsoft\*'} |
  ForEach-Object {
    [PSCustomObject]@{
      Name      = $_.TaskName
      Execute   = $_.Actions.Execute
      Args      = $_.Actions.Arguments
      User      = $_.Principal.UserId
    }
  } | Format-List

# Tasks created or modified in last 7 days
Get-ScheduledTask |
  Where-Object {$_.Date -gt (Get-Date).AddDays(-7)} |
  Select TaskName, Date
```

What If: Task pointing to temp/appdata/public directory

■ Any task executing from C:\Windows\Temp, C:\Users\Public, or %AppData% is malicious. Disable immediately then delete:

```
Disable-ScheduledTask -TaskName 'SuspiciousTaskName'
Unregister-ScheduledTask -TaskName 'SuspiciousTaskName' -Confirm:$false
```

1.2 — Registry Autorun Keys

```
$keys = @(
  'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run',
  'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce',
  'HKCU:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run',
  'HKCU:\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce',
  'HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon',
  'HKLM:\SYSTEM\CurrentControlSet\Services'
)
foreach ($k in $keys) {
  Write-Host "`n=== $k ===" -ForegroundColor Yellow
  Get-ItemProperty -Path $k -ErrorAction SilentlyContinue
}
```

```
# CRITICAL: Winlogon Shell and Userinit should ONLY be these values
# Shell = explorer.exe
# Userinit = C:\Windows\system32\userinit.exe,
Get-ItemProperty 'HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon' |
  Select Shell, Userinit
```

What If: Shell or Userinit has extra entries

■ Anything appended to Userinit or Shell beyond the defaults is a backdoor. Remove it:

```
Set-ItemProperty 'HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon' -Name 'Shell' -Value 'explorer.exe'
Set-ItemProperty 'HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon' -Name 'Userinit' -Value 'C:\Windows\system32\userinit.exe, '
```

1.3 — Malicious Services

```
# Services running from non-standard paths
Get-WmiObject Win32_Service |
  Where-Object {
    $_.PathName -notlike '*system32*' -and
    $_.PathName -notlike '*Program Files*' -and
    $_.PathName -notlike '*Microsoft*'
  } | Select Name, DisplayName, PathName, State

# Services from temp or user directories
Get-WmiObject Win32_Service |
  Where-Object {
    $_.PathName -like '*temp*' -or
    $_.PathName -like '*appdata*' -or
    $_.PathName -like '*public*' -or
    $_.PathName -like '*downloads*'
  } | Select Name, PathName
```

What If: Suspicious service found

```
Stop-Service -Name 'SuspiciousService' -Force
Set-Service -Name 'SuspiciousService' -StartupType Disabled
# Then delete the binary it points to if confirmed malicious
```

1.4 — WMI Subscriptions (Stealthiest Persistence)

```
# These survive reboots and are invisible to most admins
# Should return EMPTY on a clean machine
Get-WMIObject -Namespace root\subscription -Class __EventFilter |
  Select Name, Query

Get-WMIObject -Namespace root\subscription -Class __EventConsumer |
  Select Name, CommandLineTemplate, ScriptText

Get-WMIObject -Namespace root\subscription -Class __FilterToConsumerBinding |
  Select Filter, Consumer
```

What If: WMI subscription found

■ Any result here is almost certainly Red Team persistence. Remove all three objects:

```
# Remove in this order: binding first, then consumer, then filter
Get-WMIObject -Namespace root\subscription -Class __FilterToConsumerBinding | Remove-WMIObject
Get-WMIObject -Namespace root\subscription -Class __EventConsumer | Remove-WMIObject
Get-WMIObject -Namespace root\subscription -Class __EventFilter | Remove-WMIObject
```

1.5 — AD-Specific Persistence

```
# AdminSDHolder backdoor - persistent admin rights every 60 min
$acl = Get-ACL 'AD:\CN=AdminSDHolder,CN=System,DC=chefops,DC=local'
$acl.Access |
  Where-Object {
    $_.IdentityReference -notlike '*Domain Admins*' -and
    $_.IdentityReference -notlike '*Enterprise Admins*' -and
    $_.IdentityReference -notlike '*SYSTEM*' -and
    $_.IdentityReference -notlike '*Administrators*'
  } | Select IdentityReference, ActiveDirectoryRights

# DCSync rights - allows dumping all password hashes
$dn = (Get-ADDomain).DistinguishedName
$acl2 = Get-ACL "AD:\$dn"
$acl2.Access | Where-Object {
  $_.ObjectType -eq '1131f6aa-9c07-11d1-f79f-00c04fc2dcd2' -or
  $_.ObjectType -eq '1131f6ab-9c07-11d1-f79f-00c04fc2dcd2'
} | Select IdentityReference, ObjectType
```

```
# KRBTGT password age - if old, Red Team may have Golden Ticket
Get-ADUser krbtgt -Properties PasswordLastSet | Select Name, PasswordLastSet

# Reset KRBTGT TWICE (10 min apart) to invalidate Golden Tickets
Set-ADAccountPassword -Identity krbtgt `
  -NewPassword (ConvertTo-SecureString "$(New-Guid)$(New-Guid)" -AsPlainText -Force)
# WAIT 10 MINUTES THEN RUN AGAIN
Set-ADAccountPassword -Identity krbtgt `
  -NewPassword (ConvertTo-SecureString "$(New-Guid)$(New-Guid)" -AsPlainText -Force)
```

■ Resetting KRBTGT twice with a 10-minute gap invalidates all existing Kerberos tickets including Golden Tickets. This will briefly cause authentication errors — coordinate with your team before doing this.

Phase 2 — Account and Group Hardening

2.1 — Audit All Privileged Groups

```
$groups = @(
    'Domain Admins','Enterprise Admins','Schema Admins',
    'Backup Operators','Account Operators','DNSAdmins',
    'Group Policy Creator Owners','Server Operators'
)
foreach ($g in $groups) {
    Write-Host "`n=== $g ===" -ForegroundColor Red
    Get-ADGroupMember -Identity $g -Recursive |
        Select Name, SamAccountName, objectClass
}
```

What If: Unrecognized account in Domain Admins

```
# Remove immediately
Remove-ADGroupMember -Identity 'Domain Admins' -Members 'rogueuser' -Confirm:$false
# Then disable the account
Disable-ADAccount -Identity 'rogueuser'
# Document for IR report
```

2.2 — Find Rogue and Suspicious Accounts

```
# Accounts created in last 24 hours
Get-ADUser -Filter * -Properties Created, LastLogonDate, Enabled |
    Where-Object {$_.Created -gt (Get-Date).AddDays(-1)} |
    Select Name, SamAccountName, Created, Enabled

# Accounts with password never expires
Get-ADUser -Filter {PasswordNeverExpires -eq $true} -Properties PasswordNeverExpires |
    Select Name, SamAccountName

# Accounts with no password required
Get-ADUser -Filter {PasswordNotRequired -eq $true} |
    Select Name, SamAccountName

# Accounts that have never logged in
Get-ADUser -Filter * -Properties LastLogonDate |
    Where-Object {$_.LastLogonDate -eq $null -and $_.Enabled -eq $true} |
    Select Name, SamAccountName
```

2.3 — Fix ms-DS-MachineAccountQuota

```
# Check current value (default 10 = any user can join machines)
Get-ADObject -Identity ((Get-ADDomain).DistinguishedName) `
    -Properties ms-DS-MachineAccountQuota

# Set to 0 immediately
Set-ADDomain -Identity chefops.local `
    -Replace @{'ms-DS-MachineAccountQuota'='0'}
```

2.4 — AS-REP Roasting Vulnerability

```
# Find accounts vulnerable to AS-REP Roasting
Get-ADUser -Filter {DoesNotRequirePreAuth -eq $true} `
    -Properties DoesNotRequirePreAuth |
```

```
Select Name, SamAccountName

# Fix all of them
Get-ADUser -Filter {DoesNotRequirePreAuth -eq $true} |
Set-ADUser -DoesNotRequirePreAuth $false
```

2.5 — Kerberoasting Vulnerability (SPNs on User Accounts)

```
# Find user accounts with SPNs set (service accounts expected, users not)
Get-ADUser -Filter {ServicePrincipalName -ne '$null'} `
-Properties ServicePrincipalName |
Select Name, SamAccountName, ServicePrincipalName

# Remove SPN from a regular user account
Set-ADUser -Identity 'username' `
-ServicePrincipalNames @{Remove='HTTP/something.chefops.local'}

# Force AES encryption on service accounts that must have SPNs
Set-ADUser -Identity 'svcaccount' `
-KerberosEncryptionType AES256,AES128
```

2.6 — Password Policy

```
# Set strong domain password policy
Set-ADDefaultDomainPasswordPolicy -Identity 'chefops.local' `
-LockoutThreshold 5 `
-LockoutDuration '00:15:00' `
-LockoutObservationWindow '00:15:00' `
-MinPasswordLength 14 `
-PasswordHistoryCount 12 `
-ComplexityEnabled $true `
-MaxPasswordAge '90.00:00:00'

# Verify
Get-ADDefaultDomainPasswordPolicy
```

Phase 3 — Service Hardening

3.1 — Disable Dangerous Services

```
# Print Spooler - PrintNightmare - disable FIRST
Stop-Service -Name Spooler -Force
Set-Service -Name Spooler -StartupType Disabled

# Remote Registry - allows remote registry edits
Stop-Service -Name RemoteRegistry -Force
Set-Service -Name RemoteRegistry -StartupType Disabled

# Fax - no use on DC
Stop-Service -Name Fax -Force
Set-Service -Name Fax -StartupType Disabled

# IIS - verify ADFS does NOT depend on it first
# Get-Service adfssrv <- check this is running before stopping IIS
Stop-Service -Name W3SVC -Force
Set-Service -Name W3SVC -StartupType Disabled
Stop-Service -Name IISADMIN -Force
Set-Service -Name IISADMIN -StartupType Disabled
```

■ Before disabling IIS, confirm ADFS is healthy: Get-Service adfssrv should show Running. If ADFS breaks after disabling IIS, re-enable it.

```
# Disable SMBv1
Set-SmbServerConfiguration -EnableSMB1Protocol $false -Force
Get-SmbServerConfiguration | Select EnableSMB1Protocol

# Disable PowerShell v2
Disable-WindowsOptionalFeature -Online -FeatureName MicrosoftWindowsPowerShellV2Root

# Check all running services for anything unexpected
Get-Service | Where-Object {$_.Status -eq 'Running'} |
    Select Name, DisplayName, StartType | Sort DisplayName
```

3.2 — Verify ADFS is Healthy

```
Get-Service adfssrv
Get-AdfsSslCertificate
Get-AdfsProperties | Select FederationServiceName

# Test the sign-in page is reachable
Test-NetConnection -ComputerName sts.chefops.local -Port 443

# Screenshot https://sts.chefops.local/adfs/ls/idpinitiatedsignon in browser
```

Phase 4 — Firewall Configuration

4.1 — Verify Firewall is On

```
Get-NetFirewallProfile | Select Name, Enabled, DefaultInboundAction, DefaultOutboundAction

# Enable all profiles if any are off
Set-NetFirewallProfile -Profile Domain,Private,Public -Enabled True
```

4.2 — Audit Existing Rules

```
# All inbound allow rules - look for anything suspicious
Get-NetFirewallRule -Direction Inbound -Action Allow -Enabled True |
  Select DisplayName, Profile | Sort DisplayName

# Find ANY/ANY permissive rules (Red Team backdoors)
Get-NetFirewallRule -Direction Inbound -Action Allow -Enabled True |
  Where-Object {$_.Profile -eq 'Any'} |
  Select DisplayName, Profile

# Find rules added recently (sort by name for timestamp pattern)
Get-NetFirewallRule | Where-Object {$_.Action -eq 'Allow'} |
  Select DisplayName, Enabled, Direction | Sort DisplayName
```

4.3 — Block Known Bad Ports

```
# Block Telnet
New-NetFirewallRule -DisplayName 'Block Telnet' -Direction Inbound `
  -Protocol TCP -LocalPort 23 -Action Block -Profile Any

# Block FTP
New-NetFirewallRule -DisplayName 'Block FTP' -Direction Inbound `
  -Protocol TCP -LocalPort 21 -Action Block -Profile Any

# Block SMBv1 on Public profile
New-NetFirewallRule -DisplayName 'Block SMBv1 Public' -Direction Inbound `
  -Protocol TCP -LocalPort 445 -Action Block -Profile Public

# Block WMI from external
New-NetFirewallRule -DisplayName 'Block WMI External' -Direction Inbound `
  -Protocol TCP -LocalPort 135 -Action Block -Profile Public
```

4.4 — Block Suspicious IP Instantly

```
# Use when Red Team IP is identified in logs
New-NetFirewallRule -DisplayName "Block RT-$(Get-Date -Format 'HHmm') " `
  -Direction Inbound `
  -RemoteAddress `
  -Action Block -Profile Any

# Block entire subnet if needed
New-NetFirewallRule -DisplayName 'Block Subnet' `
  -Direction Inbound `
  -RemoteAddress 10.X.X.0/24 `
  -Action Block
```

Phase 5 — GPO and Logging

5.1 — PowerShell Logging GPO

Open Group Policy Management Console (gpmc.msc). Create a new GPO linked to the domain. Navigate to: Computer Configuration > Administrative Templates > Windows Components > Windows PowerShell

- Enable: Turn on Module Logging
- Enable: Turn on Script Block Logging
- Enable: Turn on PowerShell Transcription

```
# Verify GPO applied on each machine
gpresult /r /scope computer

# Force GPO refresh immediately
gpupdate /force

# Check on a remote machine
Invoke-Command -ComputerName -ScriptBlock {gpresult /r /scope computer}
```

5.2 — Audit Policy

```
auditpol /get /category:*

# Enable all critical categories
auditpol /set /subcategory:'Logon' /success:enable /failure:enable
auditpol /set /subcategory:'Account Logon' /success:enable /failure:enable
auditpol /set /subcategory:'Account Management' /success:enable /failure:enable
auditpol /set /subcategory:'DS Access' /success:enable /failure:enable
auditpol /set /subcategory:'Policy Change' /success:enable /failure:enable
auditpol /set /subcategory:'Privilege Use' /success:enable /failure:enable
```

5.3 — Restrict PowerShell Execution and Disable v2

```
# Set execution policy domain-wide via GPO
# Computer Config > Admin Templates > Windows PowerShell
# Turn on Script Execution = Allow only signed scripts

# Disable PowerShell v2 engine
Disable-WindowsOptionalFeature -Online -FeatureName MicrosoftWindowsPowerShellV2Root

# Verify v2 is gone
Get-WindowsOptionalFeature -Online -FeatureName MicrosoftWindowsPowerShellV2Root
```

5.4 — Monitor Critical Event IDs

```
# Pull last 2 hours of high-priority security events
Get-WinEvent -FilterHashtable @{
    LogName     = 'Security'
    StartTime   = (Get-Date).AddHours(-2)
    Id          = @(4720,4728,4732,4756,4625,4648,4672,4776,4624)
} | Select TimeCreated, Id, Message | Format-List
```

Event ID	Meaning	Action
4720	New user account created	Verify you created it — if not, disable and IR
4728	Added to Domain Admins	Immediate — remove if unauthorized, IR now

4732	Added to local Admins	Investigate source machine
4625	Failed logon	5+ in quick succession = brute force, block IP
4648	Explicit credential logon	Lateral movement indicator
4672	Special privileges assigned	Privilege escalation indicator
4776	Credential validation	NTLM auth — watch for anomalies

Phase 6 — OU Structure and Least Privilege

6.1 — Create OU Structure

```
New-ADOrganizationalUnit -Name 'ChefOps_Users' -Path 'DC=chefops,DC=local'
New-ADOrganizationalUnit -Name 'ChefOps_Computers' -Path 'DC=chefops,DC=local'
New-ADOrganizationalUnit -Name 'ChefOps_ServiceAccounts' -Path 'DC=chefops,DC=local'
New-ADOrganizationalUnit -Name 'ChefOps_Admins' -Path 'DC=chefops,DC=local'
New-ADOrganizationalUnit -Name 'Regular_Employees' -Path 'OU=ChefOps_Users,DC=chefops,DC=local'
New-ADOrganizationalUnit -Name 'IT_Staff' -Path 'OU=ChefOps_Users,DC=chefops,DC=local'
```

6.2 — Separate Admin Accounts for IT Staff

```
# Create -adm account for each IT person
# Their regular account has NO elevated privileges
New-ADUser -Name 'ituser-adm' `
  -SamAccountName 'ituser-adm' `
  -UserPrincipalName 'ituser-adm@chefops.local' `
  -Path 'OU=ChefOps_Admins,DC=chefops,DC=local' `
  -AccountPassword (ConvertTo-SecureString 'C0mpl3xAdm!n2026' -AsPlainText -Force) `
  -Enabled $true

# Only the -adm account goes in Domain Admins
Add-ADGroupMember -Identity 'Domain Admins' -Members 'ituser-adm'
```

6.3 — Fine-Grained Password Policy for Service Accounts

```
New-ADFineGrainedPasswordPolicy -Name 'ServiceAccount_PSO' `
  -Precedence 10 `
  -MinPasswordLength 25 `
  -ComplexityEnabled $true `
  -ReversibleEncryptionEnabled $false `
  -PasswordHistoryCount 24 `
  -MaxPasswordAge '0' `
  -LockoutThreshold 3 `
  -LockoutDuration '00:30:00' `
  -LockoutObservationWindow '00:30:00'

Add-ADFineGrainedPasswordPolicySubject `
  -Identity 'ServiceAccount_PSO' `
  -Subjects 'OU=ChefOps_ServiceAccounts,DC=chefops,DC=local'
```